11.2.1 The Monad trait

What unites `Parser`, `Gen`, `Par`, `Option`, and many of the other data types we've looked at is that they're *monads*. Much like we did with `Functor` and `Foldable`, we can come up with a Scala trait for `Monad` that defines `map2` and numerous other functions once and for all, rather than having to duplicate their definitions for every concrete data type.

In part 2 of this book, we concerned ourselves with individual data types, finding a minimal set of primitive operations from which we could derive a large number of useful combinators. We'll do the same kind of thing here to refine an *abstract* interface to a small set of primitives.

Let's start by introducing a new trait, called `Mon` for now. Since we know we want to eventually define `map2`, let's go ahead and add that.

Listing 11.2 Creating a `Mon` trait for `map2`

```
trait Mon[F[_]] {
  def map2[A,B,C](fa: F[A], fb: F[B])(f: (A,B) => C): F[C] =
    fa flatMap (a => fb map (b => f(a,b)))
}
```

This won't compile, since `map` and `flatMap` are undefined in this context.

Here we've just taken the implementation of `map2` and changed `Parser`, `Gen`, and `Option` to the polymorphic `F` of the `Mon[F]` interface in the signature.[5] But in this polymorphic context, this won't compile! We don't know *anything* about `F` here, so we certainly don't know how to `flatMap` or `map` over an `F[A]`.

What we can do is simply *add* `map` and `flatMap` to the `Mon` interface and keep them abstract. The syntax for calling these functions changes a bit (we can't use infix syntax anymore), but the structure is otherwise the same.

Listing 11.3 Adding `map` and `flatMap` to our trait

```
trait Mon[F[_]] {
  def map[A,B](fa: F[A])(f: A => B): F[B]
  def flatMap[A,B](fa: F[A])(f: A => F[B]): F[B]

  def map2[A,B,C](
      fa: F[A], fb: F[B])(f: (A,B) => C): F[C] =
    flatMap(fa)(a => map(fb)(b => f(a,b)))
}
```

We're calling the (abstract) functions `map` and `flatMap` in the `Mon` interface.

This translation was rather mechanical. We just inspected the implementation of `map2`, and added all the functions it called, `map` and `flatMap`, as suitably abstract methods on our interface. This trait will now compile, but before we declare victory and move on to defining instances of `Mon[List]`, `Mon[Parser]`, `Mon[Option]`, and so on,

[5] Our decision to call the type constructor argument `F` here was arbitrary. We could have called this argument `Foo`, `w00t`, or `Blah2`, though by convention, we usually give type constructor arguments one-letter uppercase names, such as `F`, `G`, and `H`, or sometimes `M` and `N`, or `P` and `Q`.

let's see if we can refine our set of primitives. Our current set of primitives is `map` and `flatMap`, from which we can derive `map2`. Is `flatMap` and `map` a minimal set of primitives? Well, the data types that implemented `map2` all had a `unit`, and we know that `map` can be implemented in terms of `flatMap` and `unit`. For example, on `Gen`:

```
def map[A,B](f: A => B): Gen[B] =
  flatMap(a => unit(f(a)))
```

So let's pick `unit` and `flatMap` as our minimal set. We'll unify under a single concept all data types that have these functions defined. The trait is called `Monad`, it has `flatMap` and `unit` abstract, and it provides default implementations for `map` and `map2`.

Listing 11.4 Creating our `Monad` trait

```
trait Monad[F[_]] extends Functor[F] {
  def unit[A](a: => A): F[A]
  def flatMap[A,B](ma: F[A])(f: A => F[B]): F[B]

  def map[A,B](ma: F[A])(f: A => B): F[B] =
    flatMap(ma)(a => unit(f(a)))
  def map2[A,B,C](ma: F[A], mb: F[B])(f: (A, B) => C): F[C] =
    flatMap(ma)(a => map(mb)(b => f(a, b)))
}
```

Since `Monad` provides a default implementation of `map`, it can extend `Functor`. All monads are functors, but not all functors are monads.

The name *monad*

We could have called `Monad` anything at all, like `FlatMappable`, `Unicorn`, or `Bicycle`. But *monad* is already a perfectly good name in common use. The name comes from category theory, a branch of mathematics that has inspired a lot of functional programming concepts. The name *monad* is intentionally similar to *monoid*, and the two concepts are related in a deep way. See the chapter notes for more information.

To tie this back to a concrete data type, we can implement the `Monad` instance for `Gen`.

Listing 11.5 Implementing `Monad` for `Gen`

```
object Monad {
  val genMonad = new Monad[Gen] {
    def unit[A](a: => A): Gen[A] = Gen.unit(a)
    def flatMap[A,B](ma: Gen[A])(f: A => Gen[B]): Gen[B] =
      ma flatMap f
  }
}
```

We only need to implement `unit` and `flatMap`, and we get `map` and `map2` at no additional cost. We've implemented them once and for all, for any data type for which it's possible to supply an instance of `Monad`! But we're just getting started. There are many more functions that we can implement once and for all in this manner.